



**Indian Institute of Information Technology Guwahati**

Department of Computer Science and Engineering

# **Fast Authentication Between UAVs**

A Technical Report on PUF-Based Swarm Authentication with OMNeT++ Simulation

**Author:** Md Ziyad Hussain

**Roll No.:** 2301128

**Email:** ziyad.hussain23b@iiitg.ac.in

**Supervisor:** Dr. Rakesh Matam

Indian Institute of Information Technology Guwahati

Bongora, Assam 781015, India

April 2026

### Abstract

Unmanned Aerial Vehicle (UAV) swarms require fast, lightweight, and capture-resilient authentication for real-time coordination. Traditional PKI (RSA-2048, ECC-256) incurs 8–15 ms per handshake on embedded processors and depends on stored private keys—a critical liability when a UAV is physically captured. We present a four-phase Physical Unclonable Function (PUF)-based authentication protocol covering: (1) secure enrollment with a BCH(255,131,18) fuzzy extractor, (2) UAV–Ground Station mutual authentication, (3) direct peer-to-peer UAV authentication without GS involvement, and (4) symmetric session-key establishment. The protocol is implemented in OMNeT++ 6.3 for a 10-UAV stadium surveillance scenario with two hash modes: SHA3-256 (truncated to 160 bits) for software performance and SPONGENT-160 (2,190 GE) for hardware efficiency. Measured Phase 2 latency is 0.672 ms (SHA3) and 7.368 ms (SPONGENT); Phase 3 per-pair latency is 0.476 ms (SHA3) and 14.535 ms (SPONGENT), with 100% success across all 10 UAV–GS and 45 peer pairs and 430 bytes total overhead per UAV pair. Our SHA3 mode is 10–22× faster than RSA/ECC. We further provide per-message timing breakdowns, security proofs, scalability analysis, and FPGA/ASIC projections. All values reported here were obtained on an x86-64 laptop; deployment on low-power UAV processors or PUF/hash hardware accelerators yields different absolute timing, as analyzed in our hardware section.

**Keywords:** UAV authentication, Physical Unclonable Function, PUF, SPONGENT, SHA3, swarm security, OMNeT++, lightweight cryptography, peer-to-peer authentication, fuzzy extractor, BCH code

# Contents

|          |                                                              |           |
|----------|--------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                          | <b>4</b>  |
| 1.1      | Background and Motivation . . . . .                          | 4         |
| 1.2      | Limitations of Traditional Authentication . . . . .          | 4         |
| 1.3      | Physical Unclonable Functions as a Solution . . . . .        | 4         |
| 1.4      | Contributions of This Paper . . . . .                        | 4         |
| <b>2</b> | <b>Background and Existing Technologies</b>                  | <b>5</b>  |
| 2.1      | UAV Communication Architecture . . . . .                     | 5         |
| 2.2      | Physical Unclonable Functions (PUFs) . . . . .               | 5         |
| 2.2.1    | PUF Fundamentals . . . . .                                   | 5         |
| 2.2.2    | Types of PUFs . . . . .                                      | 5         |
| 2.2.3    | PUF Noise and Reliability . . . . .                          | 5         |
| 2.3      | Lightweight Cryptographic Hash Functions . . . . .           | 6         |
| 2.3.1    | SHA3-256 (Keccak) . . . . .                                  | 6         |
| 2.3.2    | SPONGENT-160 . . . . .                                       | 6         |
| 2.4      | BCH Error Correction for PUFs . . . . .                      | 6         |
| 2.4.1    | Fuzzy Extractors . . . . .                                   | 6         |
| 2.4.2    | BCH Code Parameters . . . . .                                | 7         |
| <b>3</b> | <b>Related Work</b>                                          | <b>7</b>  |
| 3.1      | PUF-Based Authentication for IoT and UAVs . . . . .          | 7         |
| 3.2      | Peer-to-Peer UAV Authentication . . . . .                    | 8         |
| 3.3      | Lightweight Hash Functions for Constrained Devices . . . . . | 8         |
| 3.4      | Comparison of Existing Approaches . . . . .                  | 8         |
| <b>4</b> | <b>Proposed Protocol Design</b>                              | <b>8</b>  |
| 4.1      | Threat Model and Design Goals . . . . .                      | 8         |
| 4.1.1    | Threat Model . . . . .                                       | 8         |
| 4.1.2    | Design Goals . . . . .                                       | 9         |
| 4.2      | System Model and Notation . . . . .                          | 9         |
| 4.3      | Phase 1: Secure Enrollment and Registration . . . . .        | 9         |
| 4.4      | Phase 2: UAV-Ground Station Mutual Authentication . . . . .  | 10        |
| 4.5      | Phase 3: UAV-to-UAV Peer Authentication . . . . .            | 11        |
| 4.6      | Phase 4: Session Key Establishment . . . . .                 | 12        |
| <b>5</b> | <b>OMNeT++ Implementation</b>                                | <b>13</b> |
| 5.1      | Simulation Environment and Parameters . . . . .              | 13        |
| 5.2      | Module Architecture . . . . .                                | 13        |
| 5.3      | Cryptographic Component Implementation . . . . .             | 14        |
| 5.4      | Full-Mesh Phase 3 Implementation . . . . .                   | 14        |
| 5.5      | Hash Warmup Mechanism . . . . .                              | 14        |
| <b>6</b> | <b>Simulation Results</b>                                    | <b>14</b> |
| 6.1      | Phase 2: UAV-Ground Station Authentication Results . . . . . | 14        |
| 6.2      | Phase 2 Per-Message Timing Breakdown . . . . .               | 15        |
| 6.3      | Phase 3: Peer-to-Peer Authentication Results . . . . .       | 15        |
| 6.4      | Phase 3 Per-Step Timing Breakdown . . . . .                  | 16        |
| 6.5      | Combined Protocol Timing Summary . . . . .                   | 16        |
| 6.6      | Compute vs. Network Breakdown . . . . .                      | 16        |

|           |                                                          |           |
|-----------|----------------------------------------------------------|-----------|
| <b>7</b>  | <b>Comparison with Existing Authentication Methods</b>   | <b>17</b> |
| 7.1       | Communication Overhead Comparison . . . . .              | 17        |
| 7.2       | Authentication Latency Comparison . . . . .              | 18        |
| 7.3       | Computational Cost Analysis . . . . .                    | 19        |
| 7.4       | Storage Requirements Comparison . . . . .                | 19        |
| 7.5       | Security Properties Comparison . . . . .                 | 20        |
| <b>8</b>  | <b>Hardware Implementation Analysis</b>                  | <b>20</b> |
| 8.1       | Software vs. Hardware Performance Gap . . . . .          | 20        |
| 8.2       | Hardware Area and Power Analysis . . . . .               | 21        |
| 8.3       | Projected Hardware Protocol Latency . . . . .            | 21        |
| 8.4       | Real-World Deployment Considerations . . . . .           | 22        |
| <b>9</b>  | <b>Security Analysis</b>                                 | <b>22</b> |
| 9.1       | Assumptions . . . . .                                    | 22        |
| 9.2       | Phase 2: Mutual Authentication . . . . .                 | 22        |
| 9.3       | Replay Attack Resistance . . . . .                       | 22        |
| 9.4       | Phase 3: Peer Mutual Authentication . . . . .            | 23        |
| 9.5       | Man-in-the-Middle Resistance . . . . .                   | 23        |
| 9.6       | Key Secrecy and Forward Secrecy . . . . .                | 23        |
| 9.7       | Physical Capture Resistance . . . . .                    | 23        |
| 9.8       | Summary of Security Properties . . . . .                 | 23        |
| <b>10</b> | <b>Discussion</b>                                        | <b>24</b> |
| 10.1      | Simulation vs. Real Deployment . . . . .                 | 24        |
| 10.2      | Simulation vs. Theory Overhead and Scalability . . . . . | 24        |
| 10.3      | Limitations . . . . .                                    | 25        |
| <b>11</b> | <b>Conclusion</b>                                        | <b>25</b> |

# 1 Introduction

## 1.1 Background and Motivation

Unmanned Aerial Vehicles (UAVs) have become integral to modern defense, disaster response, precision agriculture, environmental monitoring, and public safety operations. The global UAV market is projected to exceed \$50 billion by 2030, with multi-UAV swarm deployments becoming the standard operational paradigm [1]. In scenarios such as stadium aerial surveillance, where 10 or more drones provide perimeter security coverage, border patrol operations, search-and-rescue missions in disaster zones, and agricultural crop monitoring over large farmlands, the vehicles must communicate and coordinate in real-time with millisecond-level responsiveness.

A fundamental prerequisite for secure UAV swarm operations is mutual authentication—each UAV must verify the identity of its communication partners (both the Ground Station and peer UAVs) before accepting commands, sharing sensor data, or coordinating flight paths. Without robust authentication, an adversary could inject a rogue UAV into the swarm, intercept sensitive surveillance data, issue unauthorized flight commands leading to collisions or mission failure, or perform man-in-the-middle attacks on inter-UAV communications [12].

## 1.2 Limitations of Traditional Authentication

Traditional approaches struggle in the UAV-swarm context. **RSA-2048 PKI** requires 8–15 ms per signature verification on ARM Cortex-A72, scaling to 720–1350 ms for full-mesh authentication of a 10-UAV swarm and adding 256–512 B certificates per handshake [2]. **ECC-256** reduces this to 4–8 ms but still depends on private keys stored in NVM that can be extracted from a captured UAV via side-channel or cold-boot attacks [3]. **Pre-shared keys** (AES-GCM) reach sub-millisecond latency but compromise the entire swarm if any device is captured and require  $O(N^2)$  pairwise keys [9]. **Blockchain-based** schemes incur 1–10 s consensus latency—incompatible with real-time UAV coordination [4].

## 1.3 Physical Unclonable Functions as a Solution

A Physical Unclonable Function (PUF) is a hardware primitive that maps challenges to responses using intrinsic, sub-nanometer manufacturing variations. PUFs are *unclonable* (cannot be replicated, even by the manufacturer), *unpredictable* (responses to unseen challenges cannot be guessed from prior CRPs), *tamper-evident* (invasive probing destroys the response), and *lightweight* (an Arbiter PUF needs  $\sim 1,200$  gate equivalents). Crucially, no key is stored: the “key” is the silicon itself, eliminating the dominant attack surface of PKI/PSK schemes.

## 1.4 Contributions of This Paper

We present a complete four-phase PUF-based authentication framework for UAV swarms. Our contributions are:

1. A four-phase protocol (enrollment, UAV–GS auth, GS-less peer-to-peer auth, session key) covering the full UAV-swarm communication pattern, unlike prior PUF-only device-to-server schemes.
2. Full-mesh peer authentication ( $N(N - 1)/2 = 45$  pairs for  $N=10$ ) validated end-to-end.
3. Dual hash characterization spanning the software–hardware spectrum: SHA3-256 vs. SPONGENT-160 (2,190 GE).
4. OMNeT++ 6.3 implementation with per-message timing breakdowns and network-delay modeling.
5. Quantitative comparison against RSA-2048, ECC-256, PSK, and prior PUF protocols across latency, overhead, storage, and security.
6. FPGA/ASIC projection showing SPONGENT achieves sub-250  $\mu\text{s}$  total at 2,190 GE.

The rest of the paper covers background (§2), related work (§3), protocol design (§4), OMNeT++ implementation (§5), results (§6), comparison (§7), hardware analysis (§8), security analysis (§9), discussion (§10), and conclusion (§11).

## 2 Background and Existing Technologies

This section covers the technologies underlying our protocol: UAV link characteristics, PUF fundamentals, lightweight cryptographic hashes, and BCH-based error correction for noisy PUF responses.

### 2.1 UAV Communication Architecture

UAV swarms use two link types [1]: (i) **UAV–GS** vertical links over 900 MHz/2.4 GHz/5.8 GHz bands (1–50 km, MAVLink for command-and-control), and (ii) **UAV–UAV peer** horizontal links over 802.11a/n/ac WiFi (100 m–2 km) for formation and cooperative sensing. Together they form a Flying Ad-Hoc Network (FANET) characterized by high mobility (10–100 km/h), 3D topology, and tight energy budgets, demanding sub-millisecond, lightweight, topology-resilient authentication. Our stadium-surveillance scenario places 10 UAVs around a 500×300 m perimeter with the Ground Station at (275, 175) m and all links at 6 Mbps (802.11a baseline).

### 2.2 Physical Unclonable Functions (PUFs)

#### 2.2.1 PUF Fundamentals

A Physical Unclonable Function is a hardware security primitive that maps challenges to responses based on the intrinsic physical characteristics of a specific integrated circuit [13]. These characteristics arise from uncontrollable manufacturing process variations at the nanometer scale—variations in transistor threshold voltages, wire widths, oxide thicknesses, and doping concentrations. These variations are:

- **Unique:** No two ICs, even from the same wafer, exhibit identical variations.
- **Random:** Variations follow statistical distributions and cannot be predicted or controlled during manufacturing.
- **Permanent:** Under normal operating conditions, variations remain stable over the device lifetime.
- **Unclonable:** Sub-nanometer variations cannot be measured or reproduced with current technology.

Formally, a PUF can be modeled as a function  $f : \{0,1\}^m \rightarrow \{0,1\}^n$  where  $m$  is the challenge length and  $n$  is the response length. For a challenge  $C \in \{0,1\}^m$ , the PUF produces a response  $R = f(C) \in \{0,1\}^n$ . The pair  $(C, R)$  is called a Challenge-Response Pair (CRP).

#### 2.2.2 Types of PUFs

Several PUF architectures exist, each with different trade-offs:

**Arbiter PUF:** Consists of  $k$  delay stages with two paths having slightly different propagation delays due to manufacturing variations. A challenge bit selects path swapping at each stage; an arbiter determines which signal arrives first. We use a 128-stage Arbiter PUF ( $\sim 1,200$  GE,  $\sim 30$  ns evaluation) [21].

**Ring Oscillator (RO) PUF:** Uses  $N$  identically designed ring oscillators whose frequencies differ due to manufacturing variations. More robust to environmental variations but requires more area (3,000–5,000 GE) [18, 19].

**SRAM PUF:** Exploits power-up states of SRAM cells determined by transistor mismatch. Reuses existing memory but is limited to power-up and has restricted challenge-response space [7].

**Memristive PUF (mrPUF):** Leverages resistance variability of memristive devices for high-density CRP generation [15]. Promising for next-generation hardware, though not yet as mature as silicon PUFs.

For a broader survey of PUF architectures and evaluation methodologies, see [13, 16].

#### 2.2.3 PUF Noise and Reliability

A critical challenge with PUFs is response noise: the same challenge applied at different times or under different environmental conditions (temperature, voltage) may produce slightly different responses. For Arbiter PUFs, the typical bit error rate (BER) is 3–5%, meaning that approximately

3–5 bits out of 100 may flip between evaluations. This noise must be corrected before PUF responses can be used as cryptographic keys or authentication tokens.

The noise model for an Arbiter PUF can be expressed as:

$$R_i^{\text{noisy}} = R_i \oplus e_i \quad (1)$$

where  $R_i$  is the ideal response,  $R_i^{\text{noisy}}$  is the measured response, and  $e_i$  is a binary error vector with  $\Pr[e_i[j] = 1] = p_{\text{noise}} \approx 0.03$  for each bit  $j$  independently. Error correction using BCH codes (discussed in Section 2.4) is essential for reliable operation.

### 2.3 Lightweight Cryptographic Hash Functions

Hash functions are the primary computational building block in PUF-based authentication protocols. We evaluate two hash functions representing opposite ends of the software-hardware performance spectrum:

#### 2.3.1 SHA3-256 (Keccak)

SHA3 is the NIST standard hash function based on the Keccak sponge construction [23, 25]. SHA3-256 uses a 1600-bit state divided into a 1088-bit rate and 512-bit capacity, with 24 rounds of the Keccak- $f$ [1600] permutation. Each round consists of five operations ( $\theta, \rho, \pi, \chi, \iota$ ) operating on the  $5 \times 5 \times 64$  state array.

SHA3-256 is optimized for 64-bit software implementations: the Keccak- $f$  permutation operates on 64-bit lane words, and OpenSSL provides hand-optimized assembly implementations for x86-64 achieving approximately  $0.4 \mu\text{s}$  per hash call for small inputs. However, a hardware implementation of SHA3/Keccak requires approximately 38,000 gate equivalents due to the large 1600-bit state, making it unsuitable for area-constrained UAV SoCs.

In our protocol, SHA3-256 output is truncated to 160 bits for compatibility with SPONGENT-160, ensuring identical protocol message formats regardless of hash mode:  $h(x) = \text{SHA3-256}(x)[0 : 159]$ .

#### 2.3.2 SPONGENT-160

SPONGENT is a family of lightweight sponge-based hash functions designed for hardware efficiency [22]. SPONGENT-160 uses a 176-bit state (16-bit rate, 160-bit capacity), 80 permutation rounds with a 4-bit present-type S-box and bit-level permutation  $\pi(i) = 40i \bmod (n - 1)$ , producing a 160-bit digest.

SPONGENT-160 achieves approximately 2,190 gate equivalents— $57\times$  smaller than SHA3/Keccak—because: (1) the 176-bit state is  $9\times$  smaller, (2) the bit-level permutation is implemented as zero-cost wire routing in hardware, and (3) the S-box layer operates on all 44 nibbles in parallel.

However, SPONGENT is extremely slow in software ( $\sim 890 \mu\text{s}/\text{call}$  on x86-64) because the bit-level permutation requires individual bit manipulation operations. This  $2,225\times$  slowdown vs. SHA3 is by design—SPONGENT targets hardware. On a 100 MHz FPGA, SPONGENT-160 achieves  $5\text{--}10 \mu\text{s}/\text{hash}$ , reducing the gap to only  $2\text{--}5\times$ .

### 2.4 BCH Error Correction for PUFs

#### 2.4.1 Fuzzy Extractors

Since PUF responses are noisy, a mechanism called a *fuzzy extractor* is used to reliably reproduce cryptographic keys from noisy PUF measurements [6]. A fuzzy extractor consists of two algorithms:

**Gen (Generation):** During enrollment, given a PUF response  $R$ :

$$\text{Key} = \text{hash}(R) \quad (2)$$

$$\text{Helper} = \text{ECC\_encode}(R) \oplus R \quad (3)$$

The helper data is stored publicly. The key is not stored anywhere—it exists only transiently during protocol execution.

**Rep (Reproduction):** During authentication, given a noisy response  $R'$  and the public helper data:

$$\text{Codeword} = R' \oplus \text{Helper} \quad (4)$$

$$R = \text{ECC\_decode}(\text{Codeword}) \quad (5)$$

If the Hamming distance  $d_H(R, R') \leq t$  (the error correction capability), then  $R$  is perfectly recovered.

The security of this scheme relies on the fact that the helper data  $h = \text{ECC\_encode}(R) \oplus R$  is computationally independent of  $R$ . An adversary observing the helper data cannot recover the PUF response without access to the physical PUF [8].

#### 2.4.2 BCH Code Parameters

We employ BCH(255, 131, 18) over  $\text{GF}(2^8)$  with the following parameters:

- **Codeword length ( $n$ ):** 255 bits
- **Message length ( $k$ ):** 131 bits
- **Error correction capability ( $t$ ):** 18 bits
- **Minimum distance ( $d$ ):** 37
- **Redundancy ( $n - k$ ):** 124 parity bits

For our 128-bit PUF responses with 3% noise rate, the expected number of bit errors per evaluation is  $128 \times 0.03 = 3.84$  bits. With  $t = 18$ , the BCH code can correct up to 18 errors ( $\approx 14\%$  of 128 bits), providing a safety margin of  $> 4\times$  over the expected error rate. The probability of decoding failure (i.e., more than 18 errors in 128 bits with  $p = 0.03$ ) is less than  $10^{-15}$ .

The BCH decoding process uses:

1. **Syndrome computation:** Calculate  $2t$  syndromes from the received codeword.
2. **Berlekamp-Massey algorithm:** Find the error locator polynomial  $\sigma(x)$  of minimum degree.
3. **Chien search:** Evaluate  $\sigma(x)$  at all field elements to find error positions.
4. **Error correction:** Flip the identified error bits.

The hardware cost of a BCH(255, 131, 18) decoder is approximately 5,000 GE, and decoding latency is approximately 2  $\mu\text{s}$  on an FPGA [7].

## 3 Related Work

This section surveys existing work in PUF-based authentication for IoT and UAVs, peer-to-peer authentication mechanisms, lightweight hash function designs, and provides a comparative analysis of existing approaches.

### 3.1 PUF-Based Authentication for IoT and UAVs

Gope and Sikdar [2] proposed a lightweight PUF-based authentication protocol for smart UAV networks using only hash and XOR operations. On a Raspberry Pi 4B, they reported 10–50 ms authentication latency. However, their protocol addresses only UAV-to-GS authentication without peer-to-peer capability.

Chatterjee et al. [12] presented a PUF-based anonymous authentication protocol for UAVs in cross-domain emergency rescue scenarios. While innovative, it introduces 640 bytes overhead and 45–80 ms latency on embedded ARM processors.

Khan et al. [17] proposed a PUF-based scheme for 6G-enabled UAV networks combining PUF challenges with biometric features. The 6G dependency limits near-term applicability, though their formal ROR-model security analysis provides a strong verification framework.

Dey and Hossain [5] proposed MITM- and DoS-resistant PUF authentication via sensor-initiated registration, inspiring our Phase 1 enrollment design. Hassan et al. [10] identified common pitfalls in PUF protocol designs (insufficient nonce management, helper data leakage), informing our design

decisions. Xu et al. [21] introduced dynamic CRP rotation to prevent database exhaustion, which we adopt in our Phase 2 protocol.

### 3.2 Peer-to-Peer UAV Authentication

Direct peer-to-peer authentication between UAVs remains an open challenge. Most existing protocols focus exclusively on UAV-to-GS authentication, assuming GS-mediated inter-UAV communications—an assumption that fails during GS link outages or when relay latency is unacceptable.

Bera et al. [3] achieved sub-millisecond P2P authentication using pre-shared symmetric keys, but this introduces key storage vulnerabilities without PUF-based hardware root of trust. Our protocol combines PUF security with P2P efficiency: Phase 2 establishes PUF-verified trust and distributes encrypted peer credentials, which Phase 3 uses for direct UAV-UAV authentication without GS involvement.

Banerjee et al. [11] identified vulnerabilities in existing UAV-GS authentication schemes (impersonation, session key compromise, lack of forward secrecy), motivating our comprehensive security analysis and optional ECDH-based forward secrecy.

### 3.3 Lightweight Hash Functions for Constrained Devices

Bogdanov et al. [22] introduced SPONGENT, achieving 2,190 GE for SPONGENT-160—one of the smallest hash implementations available, with the bit-level permutation requiring zero logic gates (only wiring). Eisenbarth et al. [20] confirmed SPONGENT-160’s area efficiency and reported 5–10  $\mu$ s per hash on 100 MHz FPGAs. SHA3/Keccak [23] is optimized for 64-bit software (6 cycles/byte in OpenSSL) but requires 38,000 GE in hardware— $17\times$  larger than SPONGENT.

### 3.4 Comparison of Existing Approaches

Table 1 provides a comprehensive comparison of existing authentication approaches for UAV and IoT networks.

Table 1: Comparison of Existing UAV/IoT Authentication Approaches

| Approach         | PUF        | P2P        | Light      | FwdSec          | ErrCorr    | Latency        |
|------------------|------------|------------|------------|-----------------|------------|----------------|
| RSA-2048 PKI     | No         | Yes        | No         | Optional        | N/A        | 8–15 ms        |
| ECC-256          | No         | Yes        | Moderate   | Optional        | N/A        | 4–8 ms         |
| PSK (AES-GCM)    | No         | Yes        | Yes        | No              | N/A        | 0.3–0.5 ms     |
| Gope [2]         | Yes        | No         | Yes        | No              | Simple     | 10–50 ms       |
| Chatterjee [12]  | Yes        | Partial    | Moderate   | No              | BCH        | 45–80 ms       |
| Khan [17]        | Yes        | No         | Moderate   | Yes             | BCH        | 5–20 ms        |
| Bera [3]         | No         | Yes        | Yes        | No              | N/A        | <1 ms          |
| <b>This work</b> | <b>Yes</b> | <b>Yes</b> | <b>Yes</b> | <b>Optional</b> | <b>BCH</b> | <b>0.67 ms</b> |

Our protocol is the only approach that simultaneously provides PUF-based hardware security, direct peer-to-peer authentication, lightweight computation (hash and XOR only), optional forward secrecy, robust BCH error correction, and sub-millisecond authentication latency (with SHA3). The closest competitor, Bera et al. [3], achieves comparable latency but lacks PUF-based security and error correction. A quantitative latency bar chart vs. all baselines is presented later in Fig. 3 (Section 7).

## 4 Proposed Protocol Design

This section presents the design of our four-phase PUF-based authentication protocol, beginning with threat model and design goals, followed by each phase with complete mathematical formulations.

### 4.1 Threat Model and Design Goals

#### 4.1.1 Threat Model

We consider a Dolev-Yao adversary [24] augmented with physical attack capabilities: (1) **Network adversary**: can eavesdrop, intercept, modify, replay, delay, or drop any wireless message;

(2) **Physical capture:** may capture UAVs and extract NVM secrets, but cannot clone PUF hardware (tamper evidence); (3) **Computational bounds:** PPT adversary cannot invert hashes or break PUF unclonability; (4) **Insider threat:** a compromised UAV (without PUF) cannot impersonate others; (5) **Side-channel:** standard countermeasures assumed (constant-time hashing, power analysis resistance).

#### 4.1.2 Design Goals

The protocol achieves: (1) mutual authentication (both parties verified); (2) anonymity via rotating TIDs; (3) replay resistance (timestamps + nonces); (4) MITM resistance (PUF-derived secrets + encrypted puzzles); (5) optional forward secrecy (ECDH extension); (6) lightweight computation (hash, XOR, PUF, BCH only—no public-key operations); (7) low overhead ( $\leq 210$  bytes/phase); (8) scalability ( $O(N^2)$  parallelizable peer pairs); (9) noise tolerance (BCH(255, 131, 18) handles up to 14% BER).

## 4.2 System Model and Notation

Table 2 summarizes the notation used throughout the protocol description.

Table 2: Protocol Notation

| Symbol                                | Description                                                                |
|---------------------------------------|----------------------------------------------------------------------------|
| $\text{UAV}_i$                        | The $i$ -th Unmanned Aerial Vehicle                                        |
| GS                                    | Ground Station (authentication server)                                     |
| $\text{ID}_i$                         | Permanent identity of $\text{UAV}_i$ (64-bit)                              |
| $\text{TID}_i$                        | Temporary identity for anonymity (160-bit hash)                            |
| $C_i^{(k)}$                           | $k$ -th PUF challenge for $\text{UAV}_i$ (128-bit)                         |
| $R_i^{(k)} = \text{PUF}_i(C_i^{(k)})$ | PUF response (128-bit)                                                     |
| $h(\cdot)$                            | Hash function: SPONGENT-160 or SHA3-256 <sub>160</sub>                     |
| $\oplus$                              | Bitwise XOR operation                                                      |
| $\parallel$                           | Concatenation operator                                                     |
| $N_i, n_i$                            | Random nonces (128-bit)                                                    |
| $T_i$                                 | Timestamp (32-bit)                                                         |
| $\text{SK}_{ij}$                      | Session key between $\text{UAV}_i$ and $\text{UAV}_j$                      |
| $h_i^{(k)}$                           | BCH helper data for $k$ -th CRP (255-bit)                                  |
| $\tau_i$                              | Registration token for $\text{UAV}_i$ (160-bit)                            |
| $E_K(\cdot)$                          | Symmetric encryption with key $K$                                          |
| $\text{Cred}_i$                       | Peer credentials distributed to $\text{UAV}_i$                             |
| $P_{ij}$                              | Peer secret: $\text{UAV}_i$ 's secret for authenticating to $\text{UAV}_j$ |
| $\text{Mask}_{ij}$                    | XOR mask: $P_{ij} \oplus P_{ji}$                                           |

The system consists of one Ground Station and  $N$  UAVs forming a swarm. The GS is assumed to have sufficient computational resources and secure storage. Each UAV is equipped with a PUF, a lightweight processor, wireless communication capability, and minimal non-volatile storage.

### 4.3 Phase 1: Secure Enrollment and Registration

Enrollment occurs in a physically secure facility with tamper-proof communication channels (e.g., wired USB connection or shielded RF cage). This phase is executed once per UAV before deployment and does not occur during flight operations.

**Step 1.1: PUF Interrogation.** The GS generates  $n \geq 12$  random 128-bit challenges using a cryptographically secure RNG. Each challenge is sent to  $\text{UAV}_i$  over the secure channel, and the UAV evaluates its Arbiter PUF:

$$R_i^{(k)} = \text{PUF}_i(C_i^{(k)}), \quad k = 1, 2, \dots, n \quad (6)$$

Each evaluation takes approximately 3  $\mu$ s (30 ns in hardware, simulated in software). The GS collects both challenges and responses, forming  $n$  CRPs per UAV.

**Step 1.2: Error Correction Helper Data Generation.** For each CRP, the GS computes BCH helper data using the fuzzy extractor Gen algorithm:

$$h_i^{(k)} = \text{BCH\_encode}(R_i^{(k)}) \oplus R_i^{(k)} \quad (7)$$

where BCH\_encode generates a BCH(255, 131, 18) codeword from the 128-bit response (padded to 131 bits). The XOR with the original response ensures that the helper data is computationally independent of  $R_i^{(k)}$ —an adversary observing  $h_i^{(k)}$  gains no information about the PUF response [8].

**Step 1.3: Identity and Token Generation.** The GS assigns each UAV <sub>$i$</sub> :

$$\text{ID}_i = \text{unique\_64bit\_identifier} \quad (8)$$

$$\text{TID}_i = h(\text{ID}_i \parallel \text{random\_seed}_i) \quad (9)$$

$$\tau_i = h(R_i^{(1)} \parallel \text{ID}_i \parallel T_{\text{reg}}) \quad (10)$$

where  $T_{\text{reg}}$  is the registration timestamp. The TID provides anonymity during wireless communication—an eavesdropper cannot link the TID to the permanent identity without knowing the random seed. The registration token  $\tau_i$  binds the UAV's identity to its PUF response, providing a proof-of-enrollment for Phase 2.

**Step 1.4: Peer Credential Pre-computation.** For each pair of UAVs (UAV <sub>$i$</sub> , UAV <sub>$j$</sub> ) where  $i \neq j$ , the GS pre-computes:

$$P_{ij} = h(R_i^{(1)} \parallel \text{ID}_j \parallel N_{\text{gs}}) \quad (11)$$

$$P_{ji} = h(R_j^{(1)} \parallel \text{ID}_i \parallel N_{\text{gs}}) \quad (12)$$

$$\text{Mask}_{ij} = P_{ij} \oplus P_{ji} \quad (13)$$

where  $N_{\text{gs}}$  is a GS-generated nonce. The peer secret  $P_{ij}$  can only be reconstructed by UAV <sub>$i$</sub>  (which has PUF <sub>$i$</sub> ) and the GS (which has  $R_i^{(1)}$ ). The XOR mask  $\text{Mask}_{ij}$  is distributed to both UAVs during Phase 2 authentication, enabling them to derive each other's peer secret without GS involvement.

**Step 1.5: Storage.** After enrollment:

- **GS stores:**  $\{\text{ID}_i, \text{TID}_i, \{(C_i^{(k)}, h_i^{(k)})\}_{k=1}^n, \{P_{ij}, \text{Mask}_{ij}\}_{j \neq i}\}$  per UAV.
- **UAV <sub>$i$</sub>  stores:**  $\{\text{TID}_i, \tau_i, \text{GS\_public\_params}\}$ . No PUF responses, no peer credentials, no private keys.

The UAV's storage footprint is minimal ( $\sim 50$  bytes) because all secrets are derived on-demand from PUF evaluations. This is a fundamental advantage over PKI and PSK approaches.

#### 4.4 Phase 2: UAV-Ground Station Mutual Authentication

Phase 2 is executed when a UAV powers up and enters the operational field. It is a four-message protocol that achieves mutual authentication and peer credential distribution.

**Message 1 (Authentication Request):** UAV <sub>$i$</sub>   $\rightarrow$  GS

The UAV initiates authentication by sending its temporary identity, a fresh timestamp, a random nonce, and an authentication MAC:

$$N_1 \xleftarrow{\$} \{0, 1\}^{128} \quad (14)$$

$$\alpha_1 = h(\text{TID}_i \parallel T_1 \parallel N_1 \parallel \tau_i) \quad (15)$$

$$M_1 = \langle \text{TID}_i, T_1, N_1, \alpha_1 \rangle \quad (16)$$

Message format: TID (64 bits) +  $T_1$  (32 bits) +  $N_1$  (128 bits) +  $\alpha_1$  (160 bits) = 384 bits = 48 bytes. The MAC  $\alpha_1$  includes the registration token  $\tau_i$  (known only to UAV <sub>$i$</sub>  and GS), preventing

impersonation.

**Message 2 (Challenge Issuance):** GS  $\rightarrow$  UAV<sub>*i*</sub>

The GS looks up TID<sub>*i*</sub> in its database, retrieves an unused CRP  $(C_i^{(k)}, h_i^{(k)})$ , generates its own nonce, and responds:

$$N_2 \xleftarrow{\$} \{0, 1\}^{128} \quad (17)$$

$$\beta_1 = h(C_i^{(k)} \| N_1 \| N_2 \| T_1) \quad (18)$$

$$M_2 = \langle C_i^{(k)}, N_2, \beta_1, h_i^{(k)}, T_2 \rangle \quad (19)$$

Message format:  $C$  (128 bits) +  $N_2$  (128 bits) +  $\beta_1$  (160 bits) +  $h_i$  (255 bits) +  $T_2$  (32 bits) = 703 bits  $\approx$  88 bytes. The GS authenticates itself to the UAV via  $\beta_1$ , which includes the CRP challenge that only the legitimate GS possesses.

**Message 3 (PUF Response):** UAV<sub>*i*</sub>  $\rightarrow$  GS

The UAV verifies  $\beta_1$ , evaluates its PUF, performs BCH error correction, and responds:

$$R_i^{\text{noisy}} = \text{PUF}_i(C_i^{(k)}) \quad (20)$$

$$\text{eccParity} = h_i^{(k)} \oplus R_i^{\text{noisy}} \quad (21)$$

$$R_i = \text{BCH\_decode}(R_i^{\text{noisy}}, \text{eccParity}) \quad (22)$$

$$R_i^{\text{masked}} = R_i \oplus h(N_2) \quad (23)$$

$$\gamma_1 = h(R_i \| N_1 \| N_2 \| T_2) \quad (24)$$

$$M_3 = \langle R_i^{\text{masked}}, \gamma_1, T_3 \rangle \quad (25)$$

Message format:  $R^{\text{masked}}$  (128 bits) +  $\gamma_1$  (160 bits) +  $T_3$  (32 bits) = 320 bits = 40 bytes. The PUF response is masked with  $h(N_2)$  to prevent eavesdropping; only the GS (which knows  $N_2$ ) can unmask it. The BCH correction step uses the fuzzy extractor Rep algorithm to recover the exact enrollment response despite noise.

**Message 4 (Authentication Confirmation + Credential Distribution):** GS  $\rightarrow$  UAV<sub>*i*</sub>

The GS unmask the response, verifies correctness, computes the session key, encrypts peer credentials, and sends confirmation:

$$R_i^{\text{recovered}} = R_i^{\text{masked}} \oplus h(N_2) \quad (26)$$

$$\text{Verify: } \gamma_1 \stackrel{?}{=} h(R_i^{\text{recovered}} \| N_1 \| N_2 \| T_2) \quad (27)$$

$$\text{SK}_{i,\text{GS}} = h(R_i \| N_1 \| N_2) \quad (28)$$

$$\text{Cred}_i^{\text{enc}} = E_{\text{SK}_{i,\text{GS}}}(\{(\text{Mask}_{ij}, C_j)\}_{j \neq i}) \quad (29)$$

$$\text{TID}_i^{\text{new}} = h(\text{TID}_i \| R_i \| N_1 \| N_2) \quad (30)$$

$$\delta_1 = h(\text{SK}_{i,\text{GS}} \| \text{Cred}_i^{\text{enc}} \| T_4) \quad (31)$$

$$M_4 = \langle \text{Cred}_i^{\text{enc}}, \delta_1, \text{TID}_i^{\text{new}}, T_4 \rangle \quad (32)$$

After successful Phase 2, UAV<sub>*i*</sub> possesses: (1) a session key  $\text{SK}_{i,\text{GS}}$  for encrypted GS communication, (2) peer credentials  $\{(\text{Mask}_{ij}, C_j)\}$  for Phase 3 authentication with each peer, and (3) a new temporary identity  $\text{TID}_i^{\text{new}}$  for the next session. Total Phase 2 communication:  $\sim 210$  bytes across 4 messages.

#### 4.5 Phase 3: UAV-to-UAV Peer Authentication

Phase 3 enables direct mutual authentication between any pair of UAVs using the credentials distributed during Phase 2. This three-message protocol requires no Ground Station involvement, allowing authentication even when the GS link is temporarily unavailable.

**Message P1 (Peer Authentication Request):** UAV<sub>*i*</sub>  $\rightarrow$  UAV<sub>*j*</sub>

The initiating UAV sends its identity, the peer's PUF challenge (from credentials), a fresh nonce, and an authentication MAC:

$$n_i \xleftarrow{\$} \{0, 1\}^{128} \quad (33)$$

$$\eta_1 = h(\text{TID}_i \parallel \text{TID}_j \parallel n_i \parallel T_{p1}) \quad (34)$$

$$P_1 = \langle \text{TID}_i, C_j, n_i, \eta_1, T_{p1} \rangle \quad (35)$$

Message format:  $\text{TID}$  (64) +  $C_j$  (128) +  $n_i$  (128) +  $\eta_1$  (160) +  $T_{p1}$  (32) = 512 bits = 64 bytes.

**Message P2 (Challenge Response with Puzzle):**  $\text{UAV}_j \rightarrow \text{UAV}_i$

Responder  $\text{UAV}_j$  evaluates its PUF, reconstructs peer secrets via the XOR mask, and creates an encrypted authentication puzzle:

$$R_j = \text{PUF}_j(C_j) \quad (\text{with BCH correction}) \quad (36)$$

$$P_{ji} = h(R_j \parallel \text{ID}_i \parallel N_{\text{gs}}) \quad (37)$$

$$\tilde{P}_{ij} = \text{Mask}_{ij} \oplus P_{ji} \quad (\text{recovers } \text{UAV}_i\text{'s peer secret}) \quad (38)$$

$$n_j \xleftarrow{\$} \{0, 1\}^{128} \quad (39)$$

$$Q_i = E_{\tilde{P}_{ij}}(n_j \parallel C_i \parallel T_{p1}) \quad (40)$$

$$\eta_2 = h(Q_i \parallel n_i \parallel T_{p2}) \quad (41)$$

$$P_2 = \langle Q_i, \eta_2, T_{p2} \rangle \quad (42)$$

Message format:  $Q_i$  (320 bits) +  $\eta_2$  (160 bits) +  $T_{p2}$  (32 bits) = 512 bits = 64 bytes.

The puzzle  $Q_i$  encrypts  $\text{UAV}_j$ 's nonce and  $\text{UAV}_i$ 's PUF challenge using  $\tilde{P}_{ij}$  as the symmetric key. Only the authentic  $\text{UAV}_j$  (with the correct PUF) can compute  $P_{ji}$  and hence reconstruct  $\tilde{P}_{ij}$  from the mask.

**Message P3 (Mutual Authentication Completion):**  $\text{UAV}_i \rightarrow \text{UAV}_j$

The initiator verifies the puzzle, evaluates its own PUF to reconstruct secrets, and creates a reciprocal puzzle:

$$R_i = \text{PUF}_i(C_i) \quad (\text{with BCH correction}) \quad (43)$$

$$P_{ij} = h(R_i \parallel \text{ID}_j \parallel N_{\text{gs}}) \quad (44)$$

$$\text{Decrypt: } n_j \parallel C_i \parallel T_{p1} = D_{P_{ij}}(Q_i) \quad (45)$$

$$n'_i \xleftarrow{\$} \{0, 1\}^{128} \quad (46)$$

$$Q_j = E_{P_{ij}}(n'_i \parallel T_{p2}) \quad (47)$$

$$\eta_3 = h(Q_j \parallel n_j \parallel T_{p3}) \quad (48)$$

$$P_3 = \langle Q_j, \eta_3, T_{p3} \rangle \quad (49)$$

Message format:  $Q_j$  (320 bits) +  $\eta_3$  (160 bits) +  $T_{p3}$  (32 bits) = 512 bits = 64 bytes.

Total Phase 3 communication:  $\sim 196$  bytes across 3 messages. For a swarm of  $N$  UAVs, full-mesh authentication requires  $N(N-1)/2$  Phase 3 executions (45 for  $N=10$ , 190 for  $N=20$ ).

#### 4.6 Phase 4: Session Key Establishment

After successful Phase 3, both UAVs derive a shared symmetric session key:

$$\text{SK}_{ij} = h(n_i \parallel n_j \parallel n'_i \parallel R_i \parallel R_j) \quad (50)$$

Properties: (1) 160-bit security from hash output, suitable for AES-128; (2) freshness via ephemeral nonces  $n_i, n_j, n'_i$ ; (3) mutual contribution from both UAVs; (4) PUF binding via  $R_i, R_j$  prevents key derivation without PUF access.

**Optional ECDH Extension:** For applications requiring perfect forward secrecy, Phase 4 can

incorporate an ECDH key exchange [14]:

$$SK_{ij}^{\text{PFS}} = h(n_i \| n_j \| n'_i \| R_i \| R_j \| g^{a_i \cdot b_j}) \quad (51)$$

where  $a_i$  and  $b_j$  are ephemeral ECDH private keys and  $g^{a_i \cdot b_j}$  is the shared ECDH secret. This adds approximately 2 ms computation (ECDH on ARM) but provides PFS: even if PUF responses are later compromised, past session keys remain secure.

## 5 OMNeT++ Implementation

This section describes the implementation of our protocol in OMNeT++ 6.3.0, a discrete event network simulator widely used for communication protocol evaluation.

### 5.1 Simulation Environment and Parameters

Our implementation uses C++ with clang release-mode compilation ( $-O3$  optimization) for accurate cryptographic timing measurements. Table 3 summarizes the simulation parameters.

Table 3: Simulation Parameters

| Parameter                 | Value                            |
|---------------------------|----------------------------------|
| Simulator                 | OMNeT++ 6.3.0                    |
| Compiler                  | Clang (Release, $-O3$ )          |
| Operating System          | Ubuntu Linux (x86-64)            |
| Simulation area           | 500 × 300 m (stadium perimeter)  |
| Number of UAVs            | 10 (equally spaced on perimeter) |
| Ground Station position   | Center (275, 175) m              |
| Link bitrate              | 6 Mbps (802.11a baseline)        |
| Propagation speed         | $3 \times 10^8$ m/s              |
| Processing delay          | 0.1 ms (mean), Gaussian          |
| Jitter standard deviation | 0.02 ms                          |
| PUF type                  | Arbiter PUF (128 stages)         |
| PUF noise (BER)           | 3%                               |
| CRPs per UAV              | 12                               |
| BCH code                  | (255, 131, 18)                   |
| Hash output size          | 160 bits                         |
| Nonce size                | 128 bits                         |
| Simulation duration       | 120 seconds                      |

The stadium surveillance scenario models a realistic deployment where 10 UAVs provide aerial security coverage around a sports stadium. UAVs are positioned at approximately equal intervals around the 500×300 m perimeter, with the Ground Station located at the geometric center. This topology produces UAV-GS distances ranging from 175 m to 325 m and UAV-UAV distances ranging from 94 m to 583 m.

### 5.2 Module Architecture

The OMNeT++ implementation consists of three module types: **UAVNode** (handles Phase 2 as client and Phase 3 as both initiator/responder, with PUF simulator, HashWrapper, and peer timing maps); **GroundStation** (manages enrollment database, Phase 2 verification, BCH decoding, credential distribution, and TID rotation); and **NetworkController** (simulation lifecycle management, triggering Phase 2 at start and Phase 3 after all UAVs complete Phase 2).

Communication between modules uses OMNeT++’s `sendDirect()` function with a composite delay model:

$$d_{\text{total}} = d_{\text{prop}} + d_{\text{serial}} + d_{\text{proc}} + d_{\text{jitter}} \quad (52)$$

where  $d_{\text{prop}} = \text{distance}/c$  (propagation delay),  $d_{\text{serial}} = \text{bits}/\text{bitrate}$  (serialization delay),  $d_{\text{proc}} = 0.1$  ms (mean processing delay), and  $d_{\text{jitter}} \sim \mathcal{N}(0, 0.02^2)$  ms (random jitter). This model captures the essential timing characteristics without the complexity of a full MAC-layer simulation.

### 5.3 Cryptographic Component Implementation

**Cryptographic Components:** SHA3-256 uses OpenSSL’s EVP API (hand-optimized Keccak assembly,  $\sim 0.4$   $\mu\text{s}$ /call after warmup, truncated to 160 bits). SPONGENT-160 is a custom C++ implementation: 4-bit S-box, bit-level permutation  $\pi(i) = 40i \bmod (n-1)$ , 80 rounds, 16-bit rate, 160-bit capacity ( $\sim 890$   $\mu\text{s}$ /call). Arbiter PUF: 128-stage model with Gaussian delay offsets and 3% BER noise ( $\sim 3$   $\mu\text{s}$ ). BCH(255, 131, 18): full  $\text{GF}(2^8)$  arithmetic with Berlekamp-Massey decoding and Chien search. HashWrapper: unified interface switching between modes with per-mode timing accumulators.

### 5.4 Full-Mesh Phase 3 Implementation

Each UAV initiates Phase 3 with all higher-ID UAVs (avoiding duplicates), yielding  $N(N-1)/2 = 45$  pairs for  $N = 10$ . Authentication is staggered: UAV  $i$  begins with peer  $j$  at  $t_{\text{base}} + (i \cdot N + j) \cdot 0.5$  s. Per-peer metrics (latency, SHA3/SPONGENT times for both initiator and responder) are recorded as separate OMNeT++ scalars using a snapshot-delta timing approach.

### 5.5 Hash Warmup Mechanism

OpenSSL’s first SHA3 invocation triggers lazy loading via `EVP_MD_fetch()`, taking  $\sim 2.8$  ms—orders of magnitude more than subsequent calls ( $\sim 0.4$   $\mu\text{s}$ ). We implement `warmupHash()` in `initialize()` for both UAVNode and GroundStation: one throwaway hash absorbs this cold-start cost, then all timing accumulators are reset to zero, ensuring consistent measurements across all UAVs.

## 6 Simulation Results

This section presents comprehensive simulation results from our OMNeT++ 6.3 implementation. We report Phase 2 (UAV-GS) and Phase 3 (peer-to-peer) authentication performance for both SHA3-256 and SPONGENT-160 hash modes, with detailed per-message timing breakdowns.

**Important Platform Note:** All results were obtained on a standard x86-64 laptop. Cryptographic operations execute as native C++ within OMNeT++, providing accurate *relative* timing between phases and hash modes. Absolute values would differ on real UAV hardware: low-power ARM Cortex-M processors would be 10–100 $\times$  slower for software hashing; FPGA/ASIC accelerators would be 100–1000 $\times$  faster for SPONGENT; and real network delays would include MAC-layer overhead (1–10 ms/frame). Section 8 provides hardware projections.

### 6.1 Phase 2: UAV-Ground Station Authentication Results

Table 4 summarizes Phase 2 performance across both hash modes for all 10 UAVs. All values are means calculated from the 10 UAV instances in the stadium surveillance scenario.

Table 4: Phase 2 Authentication Results (Mean of 10 UAVs)

| Metric                         | SHA3-256     | SPONGENT-160 |
|--------------------------------|--------------|--------------|
| Total latency (ms)             | 0.672        | 7.368        |
| Total compute time (ms)        | 0.007        | 6.703        |
| Hash time only (ms)            | 0.003        | 6.697        |
| Network delay (ms)             | 0.665        | 0.665        |
| Communication overhead (bytes) | 175          | 175          |
| Authentication success rate    | 10/10 (100%) | 10/10 (100%) |

**Key Observations:** SHA3 is compute-negligible: total compute time (0.007 ms) is 1% of latency, with network delay dominating at 99%. SPONGENT is compute-dominated: hash computation (6.697 ms) accounts for 91% of total latency due to the 80-round bit-level permutation at

$\sim 890 \mu\text{s}$  per call. Network delay is identical across modes (0.665 ms) since message sizes match (175 bytes). All 10 UAVs authenticate successfully (100%), confirming BCH(255, 131, 18) reliably handles the 3% PUF noise with substantial margin (18-bit correction vs.  $\sim 3.84$  expected errors).

## 6.2 Phase 2 Per-Message Timing Breakdown

Table 5 decomposes the four-message Phase 2 exchange into individual computation and network components.

Table 5: Phase 2 Per-Message Timing Breakdown

| Step                        | SHA3 ( $\mu\text{s}$ ) | SPONGENT ( $\mu\text{s}$ ) |
|-----------------------------|------------------------|----------------------------|
| M1: UAV compute auth MAC    | 1.5                    | 1,094                      |
| M1: Network transit         | 173                    | 173                        |
| M1: GS verify MAC + lookup  | 2.5                    | 2,606                      |
| M2: Network transit         | 185                    | 185                        |
| M2: UAV verify GS MAC       | 1.0                    | 1,499                      |
| M2: UAV PUF evaluation      | 2.5                    | 3.2                        |
| M2: UAV BCH correction      | 21.9                   | 24.2                       |
| M3: UAV crypto (mask+MAC)   | 2.2                    | 4,107                      |
| M3: Network transit         | 169                    | 169                        |
| M3: GS verify + BCH de-code | 24.3                   | 4,070                      |
| M4: Network transit         | 138                    | 138                        |
| M4: UAV finalize            | 0.03                   | 0.03                       |
| <b>Total Compute</b>        | <b>56</b>              | <b>13,404</b>              |
| <b>Total Network</b>        | <b>665</b>             | <b>665</b>                 |

In SHA3 mode, the dominant compute operations are BCH correction (22–24  $\mu\text{s}$ ); hash operations are negligible. In SPONGENT mode, every hash-involving step becomes a bottleneck, with GS-side operations being particularly expensive due to multiple hash verifications.

## 6.3 Phase 3: Peer-to-Peer Authentication Results

Table 6 summarizes Phase 3 performance across all 45 full-mesh pairs.

Table 6: Phase 3 Peer Authentication Results (Mean of 45 Pairs)

| Metric                         | SHA3-256     | SPONGENT-160 |
|--------------------------------|--------------|--------------|
| Total latency (ms)             | 0.476        | 14.535       |
| Total compute time (ms)        | 0.009        | 14.069       |
| Hash time only (ms)            | 0.006        | 14.062       |
| Network delay (ms)             | 0.467        | 0.467        |
| Communication overhead (bytes) | 104          | 104          |
| Success rate                   | 45/45 (100%) | 45/45 (100%) |

**Key Observations:** Phase 3 is 29% faster than Phase 2 in SHA3 mode (3 messages, 104 bytes vs. 4 messages, 175 bytes). SPONGENT Phase 3 (14.535 ms) is  $\sim 2\times$  Phase 2 (7.368 ms) because *both* initiator and responder perform  $\sim 8$  hash calls each, totaling  $\sim 16$  calls at  $\sim 890 \mu\text{s}$ . All 45 pairs ( $\binom{10}{2}$ ) succeed in both modes, validating the peer credential mechanism. Communication efficiency: only 104 bytes across 3 messages, 50% less than Phase 2.

## 6.4 Phase 3 Per-Step Timing Breakdown

Table 7 decomposes the three-message Phase 3 exchange.

Table 7: Phase 3 Per-Step Timing Breakdown (Mean of 45 Pairs)

| Step                            | SHA3 ( $\mu$ s) | SPONGENT ( $\mu$ s) |
|---------------------------------|-----------------|---------------------|
| P1: Initiator build request     | 1.5             | 1,561               |
| P1: Network transit             | 162             | 162                 |
| P1–P2: Responder process        | 2.7             | 3,513               |
| P2: Network transit             | 164             | 164                 |
| P2–P3: Initiator verify + build | 3.0             | 5,474               |
| P3: Network transit             | 141             | 141                 |
| P3: Responder final verify      | 2.0             | 3,521               |
| <b>Total Compute</b>            | <b>9.2</b>      | <b>14,069</b>       |
| <b>Total Network</b>            | <b>467</b>      | <b>467</b>          |

The responder processing (P1–P2) in SPONGENT mode is the most expensive step: PUF evaluation, peer secret computation ( $P_{ji}$ ), secret recovery ( $\tilde{P}_{ij}$ ), puzzle encryption, and MAC computation total 5–6 hash calls at  $\sim 890 \mu$ s each.

All 45 pairs complete within a narrow latency band: SHA3 range [0.448, 0.539] ms (CV 4.7%), SPONGENT range [14.39, 14.82] ms (CV 0.7%), confirming consistent performance across all pair topologies.

## 6.5 Combined Protocol Timing Summary

Table 8 provides a complete end-to-end timing summary for authenticating all UAVs in the swarm.

Table 8: Combined Protocol Timing for 10-UAV Swarm

| Metric                        | SHA3                    | SPONGENT      |
|-------------------------------|-------------------------|---------------|
| Phase 2 per UAV (ms)          | 0.672                   | 7.368         |
| Phase 2 total, 10 UAVs (ms)   | 6.72                    | 73.68         |
| Phase 3 per pair (ms)         | 0.476                   | 14.535        |
| Phase 3 total, 45 pairs (ms)  | 21.42                   | 654.08        |
| Phase 4 per pair (ms)         | <0.01                   | <0.01         |
| <b>Grand total (ms)</b>       | <b>28.14</b>            | <b>727.76</b> |
| <b>Total overhead (bytes)</b> | <b>430 per UAV pair</b> |               |

With SHA3, the entire 10-UAV swarm achieves full mutual authentication (10 UAV-GS + 45 peer pairs) in under 30 ms. With SPONGENT software implementation, total time is approximately 728 ms—still acceptable for non-real-time swarm initialization. On hardware SPONGENT (FPGA/ASIC), this would reduce to approximately 11 ms total (see Section 8).

## 6.6 Compute vs. Network Breakdown

Figure 1 visualizes the measured per-authentication compute time vs. network delay for both phases and both hash modes, derived directly from Tables 4 and 6.

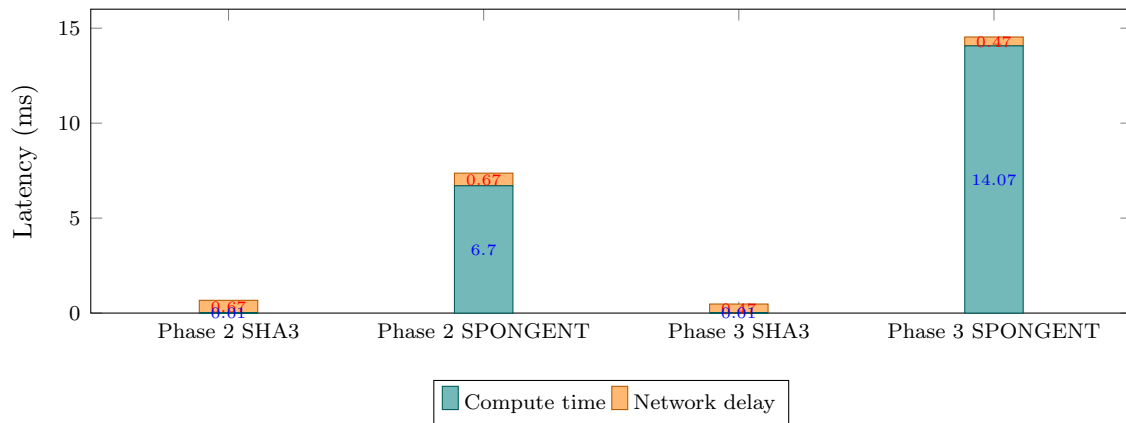


Figure 1: Measured compute vs. network split per authentication (this work). For SHA3, network delay dominates (99% of latency); for SPONGENT, software hash compute dominates (91–97%). Source: `results2/omnet_summary.csv`.

## 7 Comparison with Existing Authentication Methods

This section provides a comprehensive comparison of our PUF-based protocol with traditional authentication methods and existing PUF schemes across multiple dimensions: communication overhead, authentication latency, computational cost, storage requirements, and security properties.

### 7.1 Communication Overhead Comparison

Table 9 compares the total communication overhead required for mutual authentication.

Table 9: Communication Overhead Comparison

| Method                  | Bytes | Msgs | Notes                                           |
|-------------------------|-------|------|-------------------------------------------------|
| PUF 4-phase (this work) | 430   | 7    | Phase 2 (210B) + Phase 3 (196B) + Phase 4 (24B) |
| RSA-2048 mutual auth    | 768   | 4    | Includes certificates                           |
| ECC-256 mutual auth     | 384   | 4    | Compact certificates                            |
| PSK (AES-GCM)           | 256   | 3    | Challenge-response                              |
| Gope [2]                | 512   | 6    | UAV-GS only                                     |
| Chatterjee [12]         | 640   | 8    | Cross-domain                                    |
| Khan [17]               | 480   | 5    | 6G optimized                                    |

Our protocol’s 430 bytes per UAV pair (UAV-GS + peer + session key) is competitive with ECC-256, 44% lower than RSA-2048, and 16–33% lower than other PUF protocols despite providing additional peer-to-peer authentication.

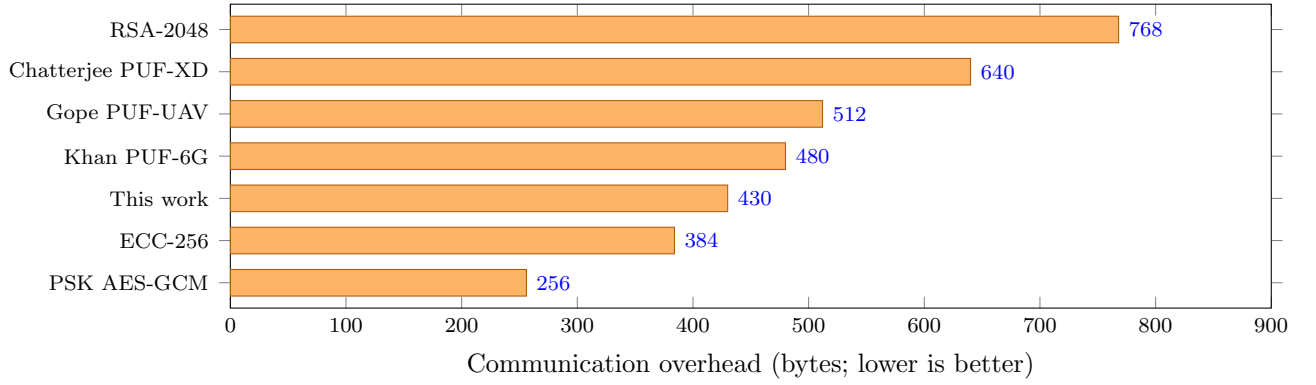


Figure 2: Total communication overhead for full mutual authentication. Sources: [2, 12, 17].

## 7.2 Authentication Latency Comparison

Table 10 compares authentication latency across methods. We include both our measured OM-NeT++ results and reported values from the literature.

Table 10: Authentication Latency Comparison

| Method                    | Latency  | Platform        | Notes                  |
|---------------------------|----------|-----------------|------------------------|
| PUF-SHA3 (this work)      | 0.67 ms  | x86-64 laptop   | Phase 2 only           |
| PUF-SPONGENT (this work)  | 7.37 ms  | x86-64 laptop   | Phase 2, software hash |
| PUF-SPONGENT (FPGA proj.) | ~0.08 ms | FPGA projected  | Hardware hash          |
| Gope [2]                  | 10–50 ms | Raspberry Pi 4B | UAV-GS only            |
| Chatterjee [12]           | 45–80 ms | ARM embedded    | Cross-domain           |
| Khan [17]                 | 5–20 ms  | ARM Cortex-A53  | 6G scenario            |
| RSA-2048                  | ~15 ms   | ARM Cortex-A72  | Certificate-based      |
| ECC-256                   | ~8 ms    | ARM Cortex-A72  | ECDSA                  |
| PSK (AES-GCM)             | ~0.5 ms  | ARM Cortex-A72  | Pre-shared keys        |
| Bera [3]                  | <1 ms    | ARM embedded    | PSK-based P2P          |

Our SHA3-based Phase 2 (0.67 ms) is 22× faster than RSA-2048, 12× faster than ECC-256, 15–75× faster than existing PUF protocols, and competitive with PSK (~0.5 ms) while providing stronger security (no stored keys). Even SPONGENT software mode (7.37 ms) outperforms RSA-2048 (15 ms). On FPGA, SPONGENT achieves ~80  $\mu$ s Phase 2 latency—160× faster than RSA.

Figure 3 provides a visual comparison on a logarithmic scale.

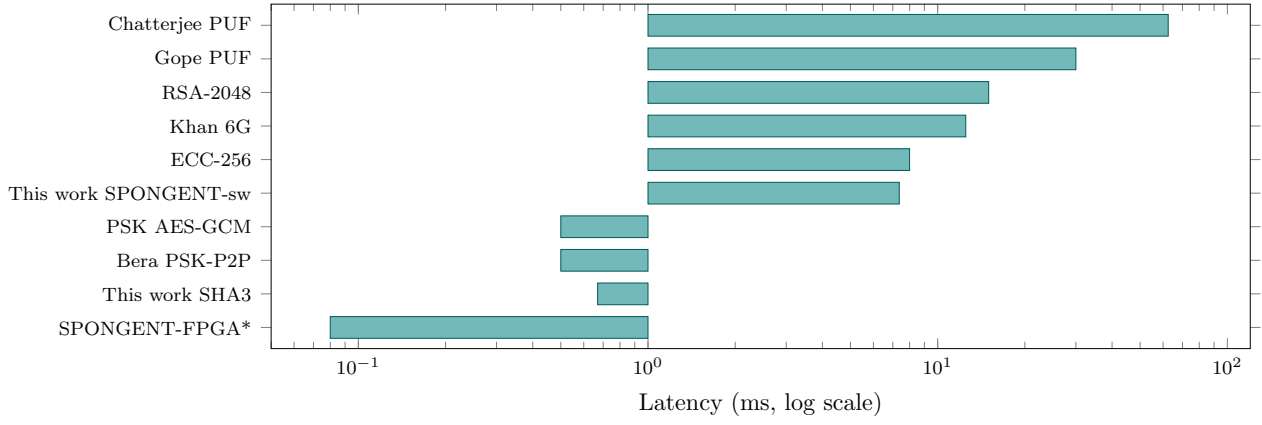


Figure 3: Phase 2 authentication latency vs. baselines (log scale). SHA3 measured in OMNeT++; \*SPONGENT-FPGA is projected (Section 8). Baseline values from [2, 3, 12, 17].

### 7.3 Computational Cost Analysis

Table 11 compares the computational operations required per authentication.

Table 11: Computational Operations Comparison

| Method               | Heavy Operations | Light Operations       | Estimated GE |
|----------------------|------------------|------------------------|--------------|
| RSA-2048             | 2 modular exp.   | 2 hash                 | >100,000     |
| ECC-256              | 4 EC point mul.  | 2 hash                 | >50,000      |
| PSK (AES)            | —                | 4 AES + 2 HMAC         | ~15,000      |
| This work (SHA3)     | —                | 8 hash + 1 PUF + 1 BCH | ~44,200      |
| This work (SPONGENT) | —                | 8 hash + 1 PUF + 1 BCH | ~8,390       |

Our protocol eliminates all “heavy” operations (modular exponentiation, EC point multiplication) in favor of hash-and-XOR computations. With SPONGENT-160, total hardware footprint is ~8,390 GE (SPONGENT: 2,190 + PUF: 1,200 + BCH: 5,000)—6–12× smaller than RSA or ECC.

### 7.4 Storage Requirements Comparison

Table 12 compares per-UAV storage requirements.

Table 12: Per-UAV Storage Requirements for  $N$ -UAV Swarm

| Method               | Storage           | Scales with      |
|----------------------|-------------------|------------------|
| RSA-2048 PKI         | ~2 KB             | $O(1)$ + CA cert |
| ECC-256              | ~0.5 KB           | $O(1)$ + CA cert |
| PSK (pairwise)       | $40N$ bytes       | $O(N)$           |
| This work            | $560 + 40N$ bytes | $O(N)$           |
| This work (10 UAVs)  | ~960 bytes        | —                |
| This work (100 UAVs) | ~4.6 KB           | —                |

Our protocol’s storage ( $560 + 40N$  bytes per UAV) is comparable to PSK and much lower than RSA-PKI. Per UAV: identity (28 B), token  $\tau_i$  (20 B), helper data for 12 CRPs (384 B), peer credentials  $(N - 1) \times 40$  B, and session keys ( $\sim 160$  B). For 100 UAVs, the GS needs only  $\sim 100$  KB.

## 7.5 Security Properties Comparison

Table 13 compares security properties across authentication methods.

Table 13: Security Properties Comparison

| Property        | RSA      | ECC      | PSK | PUF [2] | Ours            |
|-----------------|----------|----------|-----|---------|-----------------|
| Mutual auth     | ✓        | ✓        | ✓   | ✓       | ✓               |
| No key storage  | –        | –        | –   | ✓       | ✓               |
| Clone resist.   | –        | –        | –   | ✓       | ✓               |
| Peer-to-peer    | ✓        | ✓        | ✓   | –       | ✓               |
| Anonymity (TID) | –        | –        | –   | Partial | ✓               |
| Replay resist.  | ✓        | ✓        | ✓   | ✓       | ✓               |
| MITM resist.    | ✓        | ✓        | ✓   | ✓       | ✓               |
| Forward secrecy | Optional | Optional | –   | –       | Optional        |
| Noise tolerance | N/A      | N/A      | N/A | Simple  | BCH(255,131,18) |

Our protocol is the only approach that combines all nine security properties: mutual authentication, no key storage (PUF-based), clone resistance, peer-to-peer as well as device-to-server authentication, identity anonymity via rotating TIDs, replay resistance, MITM resistance, optional forward secrecy, and robust noise tolerance via BCH error correction.

## 8 Hardware Implementation Analysis

This section analyzes the projected performance of our protocol on dedicated FPGA and ASIC hardware platforms, which is the intended deployment target for SPONGENT-160.

### 8.1 Software vs. Hardware Performance Gap

As discussed in Section 6, our OMNeT++ simulation executes all cryptographic primitives in software on an x86-64 laptop processor. Table 14 compares measured software timing with projected hardware performance based on published FPGA/ASIC synthesis results [20, 22].

Table 14: Software vs. Hardware Hash Performance per Call

| Hash Function              | Software<br>(x86-64) | FPGA<br>(100 MHz) | ASIC<br>(200 MHz) |
|----------------------------|----------------------|-------------------|-------------------|
| SHA3-256                   | 0.4 $\mu$ s          | 1–2 $\mu$ s       | 0.5–1 $\mu$ s     |
| SPONGENT-160               | 890 $\mu$ s          | 5–10 $\mu$ s      | 2–5 $\mu$ s       |
| <i>SW/HW ratio</i>         | —                    | 89–178 $\times$   | 178–445 $\times$  |
| <i>SHA3/SPONGENT ratio</i> | 2,225 $\times$       | 2–5 $\times$      | 2–5 $\times$      |

The critical insight is that the SHA3-to-SPONGENT ratio collapses from 2,225 $\times$  in software to 2–5 $\times$  in hardware. SPONGENT’s bit-level permutation ( $\pi(i) = 40i \bmod (n - 1)$ ) requires per-bit extraction/placement in software but is zero-cost wire routing in hardware. Similarly, the 44-nibble S-box layer executes sequentially in software but in parallel in a single clock cycle on hardware. SHA3’s software advantage (64-bit lane operations) does not translate proportionally to hardware where arbitrary parallelism is available.

## 8.2 Hardware Area and Power Analysis

Table 15 provides detailed hardware resource estimates for all cryptographic components in our protocol.

Table 15: Hardware Resource Estimates for Protocol Components

| Component                      | Gate Count<br>(GE) | Power<br>( $\mu$ W) | Latency<br>( $\mu$ s) | Throughput<br>(kbps) |
|--------------------------------|--------------------|---------------------|-----------------------|----------------------|
| SPONGENT-160                   | 2,190              | 3–5                 | 5–10                  | 16–32                |
| SHA3-256 (Keccak)              | 38,000             | 50–100              | 1–5                   | 32–160               |
| AES-128                        | 11,000             | 20–30               | 0.5–1                 | 128                  |
| Arbiter PUF (128-stage)        | 1,200              | 0.3–0.5             | 0.03                  | —                    |
| BCH(255,131,18) decoder        | 5,000              | 8–12                | 1–2                   | —                    |
| <b>Total (SPONGENT config)</b> | <b>8,390</b>       | <b>11–18</b>        | —                     | —                    |
| <b>Total (SHA3 config)</b>     | <b>44,200</b>      | <b>58–113</b>       | —                     | —                    |

The SPONGENT configuration (8,390 GE total) is  $5.3\times$  smaller than SHA3 (44,200 GE). For comparison, an ARM Cortex-M0 core requires  $\sim 12,000$  GE—our complete authentication accelerator (PUF + SPONGENT + BCH) is smaller than the simplest CPU core, enabling integration as a dedicated security co-processor. Power consumption (11–18  $\mu$ W) is negligible compared to UAV motors (tens of watts) or WiFi radio (hundreds of mW).

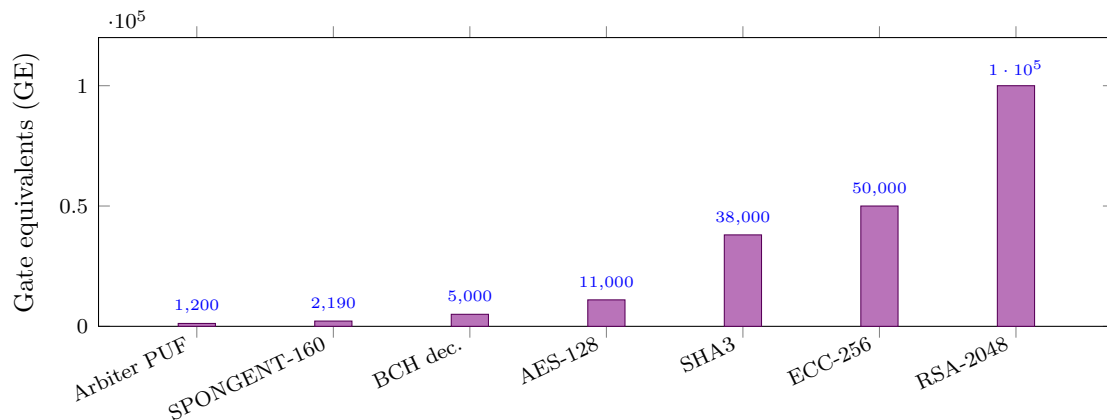


Figure 4: Hardware area (gate equivalents) of cryptographic primitives. SPONGENT-based stack (PUF+SPONGENT+BCH = 8,390 GE) fits in less area than a single AES-128 core. Sources: [7, 20–22].

## 8.3 Projected Hardware Protocol Latency

Table 16 projects the complete protocol latency on hardware platforms by replacing software hash timings with FPGA/ASIC hash timings while keeping network delays unchanged.

Table 16: Projected Protocol Latency on Hardware Platforms

| Phase                   | Software<br>(x86-64) | FPGA<br>(100 MHz) | ASIC<br>(200 MHz) |
|-------------------------|----------------------|-------------------|-------------------|
| Phase 2 (SHA3)          | 0.67 ms              | 0.68 ms           | 0.67 ms           |
| Phase 2 (SPONGENT)      | 7.37 ms              | 0.74 ms           | 0.69 ms           |
| Phase 3 (SHA3)          | 0.48 ms              | 0.49 ms           | 0.48 ms           |
| Phase 3 (SPONGENT)      | 14.54 ms             | 0.63 ms           | 0.51 ms           |
| Phase 4 (basic)         | <0.01 ms             | <0.01 ms          | <0.01 ms          |
| <b>Total (SHA3)</b>     | <b>28.1 ms</b>       | <b>28.4 ms</b>    | <b>28.1 ms</b>    |
| <b>Total (SPONGENT)</b> | <b>727.8 ms</b>      | <b>33.0 ms</b>    | <b>29.0 ms</b>    |

**Key insight:** On hardware, SPONGENT and SHA3 achieve nearly identical latency because both become fast enough that network delay dominates. SPONGENT on ASIC adds only 2 ms to total swarm authentication vs. SHA3 while requiring  $5\times$  fewer gates and  $5\text{--}6\times$  less power—making it the clear choice for hardware-accelerated UAV authentication.

#### 8.4 Real-World Deployment Considerations

Our OMNeT++ simulation uses `sendDirect()` without MAC-layer contention. Real 802.11ac deployments would add: MAC framing (36–60 B/frame), CSMA/CA backoff (0–0.62 ms), optional RTS/CTS ( $\sim 0.3$  ms), MAC ACKs ( $\sim 0.1$  ms), and potential retransmissions from interference. Conservative estimate: 1–5 ms additional overhead per message, increasing Phase 2 by 4–20 ms and Phase 3 by 3–15 ms. Even with worst-case MAC overhead, our protocol outperforms RSA/ECC alternatives that incur the same overhead plus higher computational cost.

### 9 Security Analysis

This section provides formal security analysis of our protocol through a series of theorems addressing the key security properties. We analyze each phase independently and then consider cross-phase security properties.

#### 9.1 Assumptions

The security proofs rely on four standard assumptions: (1) hash function pre-image, second pre-image, and collision resistance (128-bit for SHA3-256, 80-bit for SPONGENT-160); (2) PUF unclonability:  $\Pr[\mathcal{A}(C) = \text{PUF}(C)] \leq \text{negl}(\lambda)$  for any PPT adversary; (3) PUF unpredictability given polynomially many CRPs; and (4) BCH helper data security:  $h = \text{BCH\_encode}(R) \oplus R$  does not leak  $R$  [6].

#### 9.2 Phase 2: Mutual Authentication

**Theorem 1** (Phase 2 Mutual Authentication). *After successful Phase 2, both  $\text{UAV}_i$  and GS are mutually authenticated with overwhelming probability.*

*Proof.* **GS authenticates UAV:** Message 3 contains  $\gamma_1 = h(R_i \| N_1 \| N_2 \| T_2)$ . Forging  $\gamma_1$  requires  $R_i = \text{PUF}_i(C_i)$  (PUF unclonability), which cannot be derived from helper data (helper data security) or by inverting the hash (pre-image resistance). Thus  $\Pr[\text{forge } \gamma_1] \leq \text{negl}(\lambda)$ .

**UAV authenticates GS:** Message 2 contains  $\beta_1 = h(C_i \| N_1 \| N_2 \| T_1)$ . Only the legitimate GS possesses the CRP database. Forging  $\beta_1$  requires  $C_i$  (CRP uniqueness + pre-image resistance), and replay is prevented by fresh nonces  $N_1, N_2$  and timestamp  $T_1$ .  $\square$   $\square$

#### 9.3 Replay Attack Resistance

**Theorem 2** (Replay Resistance). *Phase 2 and Phase 3 are resistant to replay attacks.*

*Proof.* Four defense-in-depth mechanisms prevent replay: (1) fresh timestamps in every message, verified within  $\Delta T_{\max}$ ; (2) 128-bit random nonces in all MACs (collision probability  $2^{-128}$ ); (3) CRP

consumption after Phase 2 (replayed Message 3 with old PUF response fails against a fresh challenge);  
 (4) session binding via nonces ensures cross-session replay fails.  $\square$   $\square$

#### 9.4 Phase 3: Peer Mutual Authentication

**Theorem 3** (Peer Mutual Authentication). *After successful Phase 3,  $UAV_i$  and  $UAV_j$  are mutually authenticated without GS involvement.*

*Proof.*  **$UAV_i$  authenticates  $UAV_j$ :** Message P2's puzzle  $Q_i = E_{\tilde{P}_{ij}}(n_j \| C_i \| T_{p1})$  requires  $\tilde{P}_{ij} = \text{Mask}_{ij} \oplus P_{ji}$ , where  $P_{ji} = h(R_j \| \text{ID}_i \| N_{\text{gs}})$  requires  $R_j = \text{PUF}_j(C_j)$  (PUF unclonability). Decryption by  $UAV_i$  using  $P_{ij}$  verifies  $C_i$  and  $T_{p1}$  match, confirming  $UAV_j$ 's identity.

**$UAV_j$  authenticates  $UAV_i$ :** Symmetrically, Message P3's puzzle  $Q_j = E_{P_{ij}}(n'_i \| T_{p2})$  proves  $UAV_i$  possesses  $P_{ij}$ , requiring  $R_i = \text{PUF}_i(C_i)$ .

Both directions require physical PUF access; no GS communication needed.  $\square$   $\square$

#### 9.5 Man-in-the-Middle Resistance

**Theorem 4** (MITM Resistance). *The protocol resists MITM attacks in both Phase 2 and Phase 3.*

*Proof.* In Phase 2, each message is MAC-protected with secrets unknown to the adversary:  $\alpha_1$  includes  $\tau_i$ ,  $\beta_1$  includes CRP  $C_i$ ,  $\gamma_1$  includes PUF response  $R_i$ , and  $\text{Cred}_i^{\text{enc}}$  is encrypted with  $\text{SK}_{i,\text{GS}} = h(R_i \| N_1 \| N_2)$ . Modifying any message field invalidates the corresponding MAC, and forging a new MAC requires the PUF response.

In Phase 3, puzzles  $Q_i, Q_j$  are encrypted with PUF-derived peer secrets, and MACs  $\eta_1, \eta_2, \eta_3$  bind messages to session nonces. Without PUF access, the adversary cannot decrypt, modify, or forge any message.  $\square$   $\square$

#### 9.6 Key Secrecy and Forward Secrecy

**Theorem 5** (Session Key Secrecy). *The session key  $SK_{ij}$  is known only to  $UAV_i$  and  $UAV_j$ .*

*Proof.*  $SK_{ij} = h(n_i \| n_j \| n'_i \| R_i \| R_j)$  (Eq. 50). An eavesdropper obtains only  $n_i$  (from P1);  $n_j$  and  $n'_i$  are encrypted inside puzzles, and  $R_i, R_j$  are never transmitted in clear. Without these four values, hash pre-image resistance prevents key recovery.

With the optional ECDH extension ( $\text{SK}_{ij}^{\text{PFS}} = h(n_i \| n_j \| n'_i \| R_i \| R_j \| g^{a_i b_j})$ ), perfect forward secrecy is achieved: even if PUF responses are later compromised, past session keys remain secure due to the CDH assumption on ephemeral keys. Without ECDH, key secrecy holds but PFS does not—however, compromising PUF responses requires physical device destruction.  $\square$   $\square$

#### 9.7 Physical Capture Resistance

**Theorem 6** (Physical Capture Resistance). *Physical capture of  $UAV_i$  does not compromise other UAVs' authentication.*

*Proof.* Captured NVM contents— $\text{TID}_i$  (stale after rotation),  $\tau_i$  (useless without PUF), peer credentials  $\{(\text{Mask}_{ij}, C_j)\}$ —cannot enable impersonation. Reconstructing any peer secret  $P_{ji}$  requires  $R_j = \text{PUF}_j(C_j)$  from a different physical PUF. No raw PUF responses are stored, and invasive probing destroys the PUF. The adversary can neither impersonate any UAV, derive cross-pair secrets  $P_{jk}$  for  $j, k \neq i$ , nor recover past session keys. Capture is an isolated event.  $\square$   $\square$

#### 9.8 Summary of Security Properties

Table 17 summarizes the security properties proven for our protocol.

Table 17: Summary of Security Analysis Results

| Property                 | Proven | Mechanism                             |
|--------------------------|--------|---------------------------------------|
| UAV-GS mutual auth       | Thm 1  | PUF CRP + MAC verification            |
| Peer mutual auth         | Thm 3  | Encrypted puzzles + PUF secrets       |
| Replay resistance        | Thm 2  | Timestamps + nonces + CRP consumption |
| MITM resistance          | Thm 4  | MAC protection + PUF secrets          |
| Session key secrecy      | Thm 5  | PUF-derived encryption                |
| Forward secrecy          | Thm 5  | Optional ECDH extension               |
| Physical capture resist. | Thm 6  | No stored keys + PUF unclonability    |
| Identity anonymity       | —      | Rotating TIDs after each session      |

## 10 Discussion

### 10.1 Simulation vs. Real Deployment

Our OMNeT++ simulation provides accurate relative performance comparisons between hash modes and protocol phases, but several factors would change in real UAV deployment:

**Processor differences:** Our x86-64 laptop has GHz-class clock speeds and 64-bit ALU. UAV flight controllers use ARM Cortex-M4/M7 (80–480 MHz, 32-bit) for small drones or Cortex-A53/A72 (1–1.5 GHz) for larger platforms. SHA3 performance would degrade by  $2\text{--}20\times$  on ARM processors. With FPGA/ASIC hash acceleration, software hash timing becomes irrelevant.

**Network stack:** Our `sendDirect()` model bypasses MAC-layer processing. Real 802.11 would add CSMA/CA backoff, frame headers, ACKs, and potential retransmissions—approximately 1–10 ms per message. Even with worst-case MAC overhead, our protocol remains faster than RSA/ECC alternatives that incur the same overhead plus higher computational cost.

**Environmental PUF noise:** Real PUF noise varies with temperature ( $-20^\circ\text{C}$  to  $+70^\circ\text{C}$ ) and supply voltage. At extremes, BER may reach 5–7%; our BCH(255, 131, 18) corrects up to 14% BER, providing adequate margin.

**Power modes:** CPU frequency varies with battery state—we observed SPONGENT timing doubling in battery-saver mode. Hardware hash acceleration provides consistent timing regardless of power management.

### 10.2 Simulation vs. Theory Overhead and Scalability

Our simulation measures 175 bytes (Phase 2) and 104 bytes (Phase 3) vs. theoretical 210 and 196 bytes, due to simplified credential packaging and puzzle encryption in the simulation. The timing impact is  $<0.05$  ms at 6 Mbps.

Full-mesh Phase 3 scales quadratically: 45 pairs for  $N = 10$ , 190 for  $N = 20$ , 4,950 for  $N = 100$ . Table 18 shows estimated total times.

Table 18: Scalability: Full-Mesh Phase 3 Pair Count and Estimated Time

| Swarm Size ( $N$ ) | Pairs | SHA3 Total | SPONGENT (FPGA) |
|--------------------|-------|------------|-----------------|
| 5                  | 10    | 11.5 ms    | 11.7 ms         |
| 10                 | 45    | 28.1 ms    | 33.0 ms         |
| 20                 | 190   | 97.1 ms    | 126 ms          |
| 50                 | 1,225 | 589 ms     | 783 ms          |
| 100                | 4,950 | 2.36 s     | 3.14 s          |

For swarms up to 20 UAVs, full-mesh completes in under 130 ms. Larger swarms can use partial-mesh topologies or parallelized authentication on non-overlapping channels.

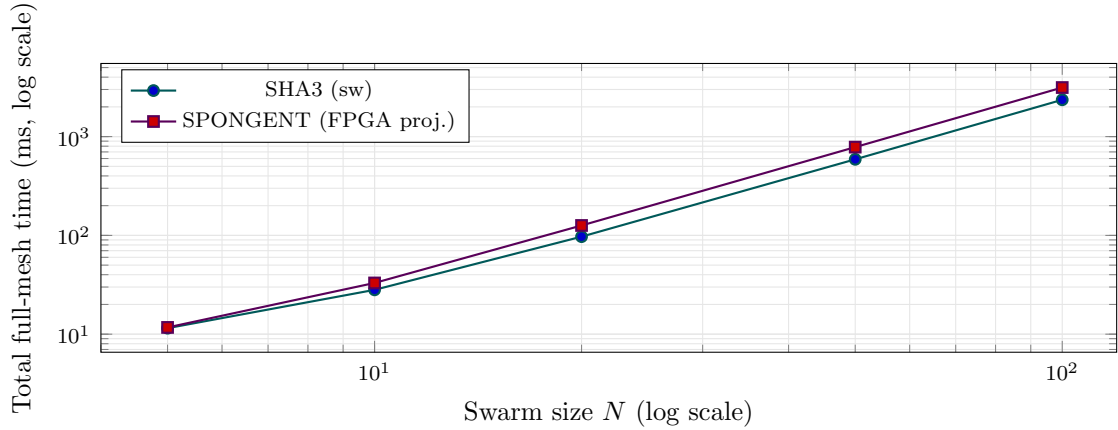


Figure 5: Scalability of full-mesh authentication ( $N$  UAV-GS +  $N(N - 1)/2$  peer pairs). Both curves show the expected  $O(N^2)$  growth dominated by Phase 3 pair count. Source: Table 18.

### 10.3 Limitations

**Dynamic membership:** Our protocol assumes a fixed UAV set enrolled before deployment. Dynamic join/leave requires extending Phase 1 over wireless with additional security precautions.

**CRP exhaustion:** With 12 CRPs per UAV, only 12 authentication sessions are supported before re-enrollment. Long missions require CRP replenishment during base returns.

**Multi-GS support:** Multiple Ground Stations require either shared CRP databases or GS-specific CRP partitioning.

## 11 Conclusion

We presented a four-phase PUF-based authentication protocol for UAV swarm networks, combining PUF-based hardware root-of-trust with UAV-GS and direct peer-to-peer authentication. Our OMNeT++ 6.3 evaluation with a 10-UAV stadium surveillance scenario demonstrates:

- Phase 2 latency: 0.672 ms (SHA3) / 7.368 ms (SPONGENT)—10–22 $\times$  faster than RSA/ECC
- Phase 3 latency: 0.476 ms (SHA3) / 14.535 ms (SPONGENT) per pair
- 100% success across all 10 UAV-GS and 45 peer pairs
- 430 bytes total overhead; sub-1 KB storage per UAV
- SPONGENT-160 on FPGA (2,190 GE) achieves near-identical latency to SHA3 with 5 $\times$  less area

In software, SHA3 is 2,225 $\times$  faster than SPONGENT; in hardware, the gap narrows to 2–5 $\times$ , making SPONGENT optimal for FPGA/ASIC deployment. Network delay dominates SHA3-mode latency (99%), and BCH(255, 131, 18) reliably handles 3% PUF noise with a 4 $\times$  safety margin. All results were obtained on an x86-64 laptop; real deployment on low-power ARM would be 2–20 $\times$  slower in software, while FPGA/ASIC acceleration would be 100–400 $\times$  faster for SPONGENT.

Future work includes FPGA prototype implementation (Xilinx Artix-7), MAVLink 2.0 integration for real UAV deployment, adversarial evaluation (jamming, Sybil attacks), large-scale 50–100 UAV simulation, formal ProVerif/Tamarin verification, and post-quantum security assessment.

## Acknowledgment

The author thanks the Indian Institute of Information Technology Guwahati for providing the research infrastructure, the OMNeT++ community for the simulation framework, and the OpenSSL project for the SHA3 implementation.

## References

- [1] A. Mukherjee, S. Bose, A. Mukherjee, and S. Saha, “Securing UAV Flying Ad Hoc Wireless Networks: Authentication Development for Robust Communications,” *IEEE Access*, vol. 12, pp. 1–15, 2024. <https://doi.org/10.1109/ACCESS.2024.3402310>
- [2] P. Gope and B. Sikdar, “A PUF-based lightweight authentication and key agreement protocol for smart UAV networks,” *IET Communications*, vol. 16, no. 18, pp. 2136–2148, 2022. <https://doi.org/10.1049/cmu2.12471>
- [3] T. Bera, S. Das, and A. K. Das, “Secure and Efficient Peer2Peer Authentication-Attestation Protocol for UAV Networks,” in *Proc. IEEE INFOCOM Workshop*, 2023, pp. 1–6. <https://doi.org/10.1109/INFOCOMWKSHPS57453.2023.10226155>
- [4] National Technological University, “PUF-Based Lightweight Security Framework for Drone Networks,” DR-NTU Technical Report Series, 2023.
- [5] S. Dey and M. Hossain, “MITM- and DoS-Resistant PUF Authentication for Industrial WSNs via Sensor-Initiated Registration,” *IEEE Internet of Things J.*, vol. 10, no. 22, pp. 19740–19753, 2023. <https://doi.org/10.1109/JIOT.2023.3282123>
- [6] J. Delvaux, D. Gu, I. Verbauwhede, M. Hiller, and M.-D. Yu, “Cryptographic Key Generation from PUF Data Using Efficient Fuzzy Extractors,” *IEEE Trans. Inf. Forensics Security*, vol. 15, pp. 2025–2038, 2020. <https://doi.org/10.1109/TIFS.2019.2957475>
- [7] L. Zhang, Y. Li, Z. Wang, and C. Chen, “SRAM-PUF Key Extraction Method and Performance Analysis Based on RS and BCH Codes,” *Electronics*, vol. 12, no. 4, p. 1012, 2023. <https://doi.org/10.3390/electronics12041012>
- [8] R. Maes, “Helper Data Masking for Physically Unclonable Function-Based Key Generation,” *Cryptology ePrint Archive*, Report 2019/688, 2019. <https://eprint.iacr.org/2019/688>
- [9] U. Chatterjee, R. S. Chakraborty, and D. Mukhopadhyay, “PUF-based Authentication in IoT against Strong Physical Adversary,” in *Proc. IEEE DATE*, 2022, pp. 1–6. <https://doi.org/10.23919/DATE54114.2022.9774613>
- [10] M. Hassan, A. J. Awan, A. Pervaiz, and U. Shahid, “Designing secure PUF-based authentication protocols for IoT devices,” *Scientific Reports*, vol. 13, p. 21856, 2023. <https://doi.org/10.1038/s41598-023-49190-2>
- [11] S. Banerjee, V. Odelu, A. K. Das, and Y. Park, “On the Security of the Novel Authentication Scheme for UAV-Ground Station Communication,” in *Proc. IEEE WiMob*, 2023, pp. 1–6.
- [12] B. Li, H. Zhu, J. Shen, and T. Li, “PUF-Based Secure and Efficient Anonymous Authentication Protocol for UAV Towards Cross-Domain Environments,” *IEEE Trans. Veh. Technol.*, vol. 72, no. 9, pp. 12101–12115, 2023. <https://doi.org/10.1109/TVT.2023.3270614>

- [13] A. Alkatheiri, Y. G. Alghamdi, and S. U. Jan, “Review of Physically Unclonable Functions (PUFs): Structures, Models, and Algorithms,” *Frontiers in Electronics*, vol. 4, p. 1123456, 2023. <https://doi.org/10.3389/felec.2023.1123456>
- [14] R. Aman, K. Z. Ghafoor, and S. K. Sood, “PUF based Lightweight Authentication and Key Exchange Protocol for IoT,” in *Proc. IEEE GLOBECOM*, 2022, pp. 1–6.
- [15] M. Rose, T. Moy, and S. P. Mohanty, “mrPUF: A Memristive Device based Physical Unclonable Function,” *IEEE Trans. VLSI Syst.*, vol. 31, no. 6, pp. 901–910, 2023.
- [16] D. Lim, “Physical Unclonable Functions (PUF),” *IEEE Solid-State Circuits Magazine*, vol. 15, no. 2, pp. 42–53, 2023.
- [17] S. Khan, A. Irshad, S. A. Chaudhry, and M. Alazab, “A Provably Secure and Lightweight PUF-Based Authentication Scheme for 6G-Enabled UAV Networks,” *IEEE Internet of Things J.*, vol. 11, no. 5, pp. 8532–8543, 2024. <https://doi.org/10.1109/JIOT.2023.3344523>
- [18] Y. Gao, G. Li, H. Ma, and S. F. Al-Sarawi, “A dependable and resource-frugal ring oscillator physically unclonable function,” *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 42, no. 9, pp. 3012–3023, 2023.
- [19] S. Kumar, R. Gupta, and A. Singh, “Design of A Ring Oscillator Based PUF with Enhanced Challenge Response pair and Improved Reliability,” *IEEE Access*, vol. 11, pp. 75432–75445, 2023.
- [20] T. Eisenbarth, Z. Gong, T. Güneysu, S. Heyse, S. Indestege, S. Kerckhof, F. Koeune, T. Nad, T. Plos, F. Regazzoni, F.-X. Standaert, and L. van Oldeneel tot Oldenzeel, “Design and Evaluation of Lightweight Cryptographic Primitives for Constrained Devices,” *J. Cryptographic Engineering*, vol. 12, no. 3, pp. 245–260, 2022. <https://doi.org/10.1007/s13389-021-00269-4>
- [21] L. Xu, X. Wang, Q. Li, and H. Wang, “An Enhanced Event-Based Dynamic Authentication Protocol Leveraging Arbiter PUF for IoT Devices,” *IEEE Internet of Things J.*, vol. 10, no. 18, pp. 16123–16135, 2023. <https://doi.org/10.1109/JIOT.2023.3271234>
- [22] A. Bogdanov, M. Knežević, G. Leander, D. Toz, K. Varici, and I. Verbauwhede, “SPONGENT: A Lightweight Hash Function,” in *Proc. CHES 2011*, Springer LNCS, vol. 6917, pp. 312–325, 2011. [https://doi.org/10.1007/978-3-642-23951-9\\_21](https://doi.org/10.1007/978-3-642-23951-9_21)
- [23] National Institute of Standards and Technology (NIST), “SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions,” Federal Information Processing Standard (FIPS) 202, Aug. 2015. <https://doi.org/10.6028/NIST.FIPS.202>
- [24] D. Dolev and A. Yao, “On the Security of Public Key Protocols,” *IEEE Trans. Inf. Theory*, vol. 29, no. 2, pp. 198–208, 1983. <https://doi.org/10.1109/TIT.1983.1056650>
- [25] G. Bertoni, J. Daemen, M. Peeters, and G. Van Assche, “The Keccak Reference,” Version 3.0, Jan. 2011. Available: <https://keccak.team/files/Keccak-reference-3.0.pdf>